

Title:

AI-Based Image Colorization: Deep Learning Improvement on Black & White Photographs

Authors:

Antra Nayudu, Kartik Kalra, Devi Tanuku

Introduction

The objective of this project is to develop a GAN-based solution that automatically colorizes black-and-white photographs. Our goals are to preserve fine detail, produce vivid and accurate color predictions, and ensure the method is scalable for both historical restoration and modern digital media applications.

Real-World Problem Overview

Manually restoring or colorizing historical and archival images is time-consuming, labor-intensive, and often requires expert artistic skill. Our AI system addresses this by predicting and applying realistic, context-aware colors at pixel level, reducing both cost and turnaround time for museums, media companies, and digital artists.

Significance And Potential Impact

1. Scalability: Automates restoration at scale for thousands of images.
2. Accessibility: Lowers the barrier for smaller institutions or individuals to preserve cultural heritage.
3. Innovation: Powers new workflows in digital art, film color grading, and interactive storytelling.
4. Quality: Maintains fine detail and produces vivid, coherent color distributions.

Current Limitations & Future Directions

Despite strong overall performance, the model still occasionally mispredicts hues on rare or context-specific objects—such as vintage cars or period costumes—and can introduce minor bleeding or desaturation in fine textures like foliage or fabric patterns.

Future Improvements

To enhance the model's fidelity, we plan to integrate self-attention layers to capture long-range dependencies and fine structures, incorporate perceptual (VGG-based) and style losses for greater semantic consistency and artistic quality, fine-tune on specialized historical archives to reduce era- or region-specific biases, and build an interactive, user-guided interface that lets editors adjust color hints in key regions.

3. Dataset Description: Dataset Summary

The primary dataset is DIV2K, sourced from Kaggle, which comprises over 800 high-resolution (2K) PNG images. These images cover a wide variety of scenes—landscapes, portraits, historic landmarks, and everyday objects—making the corpus well suited for learning complex color distributions and contextual cues.

3.2 Preprocessing Pipeline

1. **Corrupted-File Removal:** Scan and discard any images that fail to load or contain incomplete pixel data.
2. **Resolution Standardization:** Resize all images to a uniform resolution (e.g., 512×512 pixels) while preserving aspect ratio via padding.
3. **Normalization:** Scale pixel values from $[0, 255]$ to $[0, 1]$ to stabilize GAN training.
4. **Data Augmentation** Random horizontal and vertical flips to increase data diversity, And Random rotations to improve robustness to orientation.
5. **Train–Validation–Test Split:** Allocate 80% of images for training, 10% for validation, and 10% for final testing, ensuring a balanced mix of scene types in each partition

4. Methodology

Implemented Solution

We built a conditional Pix2Pix GAN in TensorFlow 2.x that colorizes 128×128 grayscale inputs. A U-Net generator downsamples to a 4×4 bottleneck and upsamples back with skip connections, while a PatchGAN discriminator evaluates 8×8 patches for realism. Training uses mixed-precision and dynamic GPU memory growth, with adversarial (BCE) and L1 reconstruction losses ($\lambda = 100$) optimized via Adam. Every 10 epochs we visualize results, and the trained generator is saved for deployment.

Build_generator

The `build_generator` implements a U-Net: it downsamples a $128 \times 128 \times 1$ grayscale input through five stride-2 conv blocks to a 4×4 bottleneck, then upsamples back to 128×128 via deconv blocks with skip connections, BatchNorm, and LeakyReLU/ReLU activations. A final 3×3 convolution with sigmoid produces the $128 \times 128 \times 3$ color output in $[0, 1]$.

```

# -----
# Generator Model
# -----
def build_generator(input_shape=(128, 128, 1)):
    inputs = Input(shape=input_shape)
    def conv_block(x, filters, kernel_size=3, bn=True):
        if bn:
            x = BatchNormalization()(x)
            x = LeakyReLU(alpha=0.2)(x)
        return x
    def deconv_block(x, skip, filters, kernel_size=3, dropout_rate=0):
        x = Conv2DTranspose(filters, kernel_size, strides=2, padding='same', activation='relu')(x)
        if dropout_rate:
            x = tf.keras.layers.Dropout(dropout_rate)(x)
        x = Concatenate()([x, skip])
        return x
    e1 = conv_block(inputs, 64, bn=False)
    e2 = conv_block(e1, 128)
    e3 = conv_block(e2, 256)
    e4 = conv_block(e3, 512)
    e5 = conv_block(e4, 512)
    b = conv2d(512, 3, padding='same', activation='relu')(e5)
    d1 = deconv_block(b, e4, 512, dropout_rate=0.5)
    d2 = deconv_block(d1, e3, 256, dropout_rate=0.5)
    d3 = deconv_block(d2, e2, 128)
    d4 = deconv_block(d3, e1, 64)
    d5 = Conv2DTranspose(3, 3, strides=2, padding='same', activation='sigmoid', dtype='float32')(d4)
    return Model(inputs, d5)

```

Build Discriminator

A PatchGAN discriminator that concatenates the 128×128×1 grayscale and 128×128×3 color inputs, applies four 4×4 stride-2 conv+LeakyReLU (with BatchNorm except first) blocks to downsample to 8×8, then outputs an 8×8 sigmoid “real vs. fake” patch map.

```

# -----
# Discriminator
# -----
def build_discriminator(input_shape_gray=(128, 128, 1), input_shape_color=(128, 128, 3)):
    input_gray = Input(shape=input_shape_gray)
    input_color = Input(shape=input_shape_color)
    combined_input = Concatenate()([input_gray, input_color])
    def disc_block(x, filters, kernel_size=4, strides=2, bn=True):
        if bn:
            x = BatchNormalization()(x)
            x = LeakyReLU(alpha=0.2)(x)
        return x
    d1 = disc_block(combined_input, 64, bn=False)
    d2 = disc_block(d1, 128)
    d3 = disc_block(d2, 256)
    d4 = disc_block(d3, 512)
    patch_out = Conv2D(1, 4, padding='same')(d4)
    patch_out = Activation('sigmoid')(patch_out)
    return Model([input_gray, input_color], patch_out)

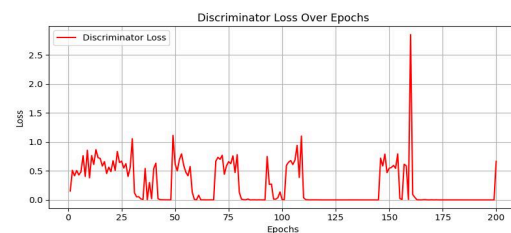
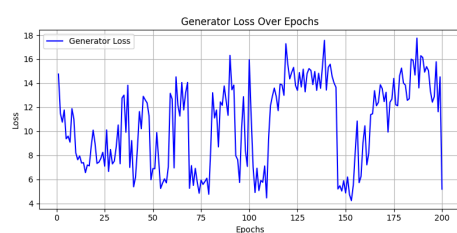
# -----
# Loss Functions & Optimizers
# -----
bce_loss = tf.keras.losses.BinaryCrossentropy(from_logits=False)
def l1_loss(y_true, y_pred):
    return tf.reduce_mean(tf.abs(y_true - y_pred))
gen_optimizer = tf.keras.optimizers.Adam(2e-4, beta_1=0.5)
disc_optimizer = tf.keras.optimizers.Adam(2e-4, beta_1=0.5)
LAMBDA = 100

```

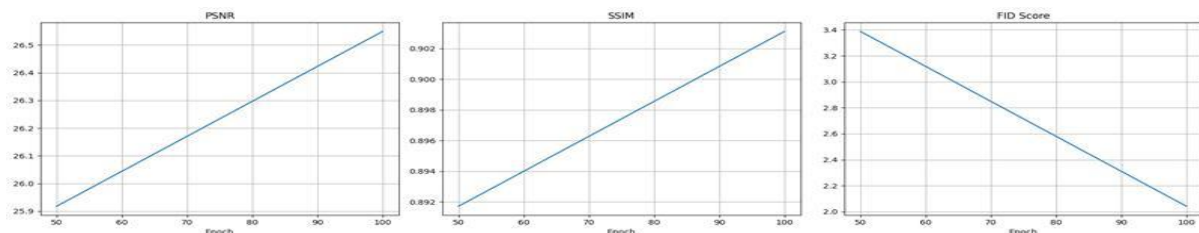
Technology stack

1. Python 3.8+: Scripting, orchestration
2. TensorFlow 2.x: Model definition, custom training loops, mixed precision
3. OpenCV (cv2): Image I/O, resizing, color-space conversion
4. NumPy: Array operations, normalization
5. Matplotlib: Training-time visualization
6. OS / os.path: File management, directory traversal
7. Keras Layers: Building conv, upsampling, normalization, activations
8. Adam Optimizer: Network optimization ($\text{lr}=2\text{e-}4$, $\beta_1=0.5$)
9. BCE & L1 Losses: Adversarial and pixel-wise objectives

Training and Validation Losses



5. Evaluation Metrics

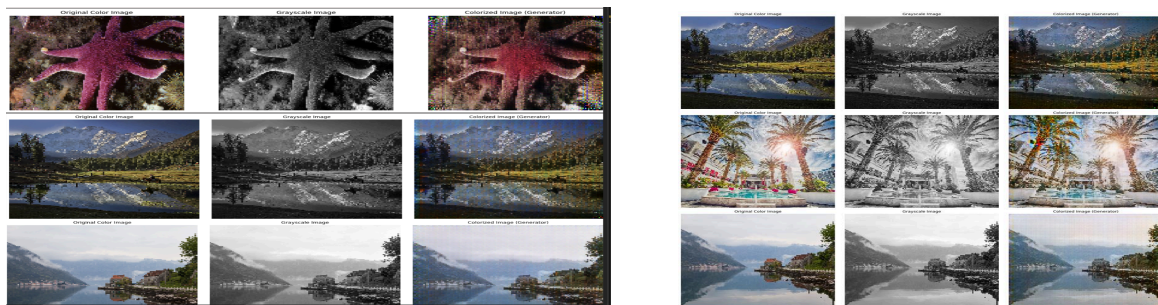


1. PSNR: Measures pixel-level accuracy; higher values indicate better image quality.
2. SSIM: Assesses perceptual similarity, focusing on luminance, contrast, and structure.
3. FID: Evaluates realism by comparing generated and real image feature distributions; lower scores are better.
4. Trends: PSNR and SSIM improve with training, while FID decreases, indicating enhanced image quality and realism.
5. Summary: These metrics collectively evaluate reconstruction accuracy (PSNR, SSIM) and realism (FID).

6. Discussion

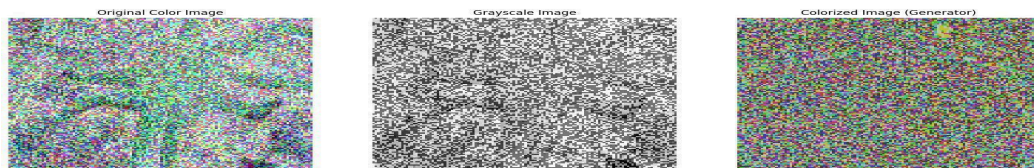
6.1 Results

Overall, the Pix2Pix colorizer delivers strong structural fidelity and plausible, vivid hues while still exhibiting minor speckle noise, color bleeding, and occasional hue inconsistencies. Promising results with room to refine via perceptual losses, attention layers, and targeted data augmentation.



6.2 Challenges & Fixes

1. Early preview executed before training: fixed by correcting execution flow and denormalization logic



2. Rainbow noise artifacts: caused by normalization errors; resolved by cleaning the grayscale pipeline and restructuring the training order



References

1. TensorFlow Developers. (2023). *TensorFlow: An end-to-end open source machine learning platform*. TensorFlow. <https://www.tensorflow.org/>
2. Chollet, F. (2015). *Keras: Deep Learning for Humans*. <https://keras.io/>
3. Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering*, 9(3), 90–95. <https://doi.org/10.1109/MCSE.2007.55>
4. Van der Walt, S., Schönberger, J. L., Nunez-Iglesias, J., Boulogne, F., Warner, J. D., Yager, N., ... & Yu, T. (2014). scikit-image: image processing in Python. *PeerJ*, 2, e453. <https://doi.org/10.7717/peerj.453>
5. Isola, P., Zhu, J. Y., Zhou, T., & Efros, A. A. (2017). Image-to-image translation with conditional adversarial networks. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 1125–1134. <https://doi.org/10.1109/CVPR.2017.632>
6. Timofte, R., Agustsson, E., Van Gool, L., Yang, M. H., & Zhang, L. (2017). NTIRE 2017 challenge on single image super-resolution: Methods and results. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition Workshops*, 114–125. Retrieved from <https://www.kaggle.com/datasets/ambarish/div2k>